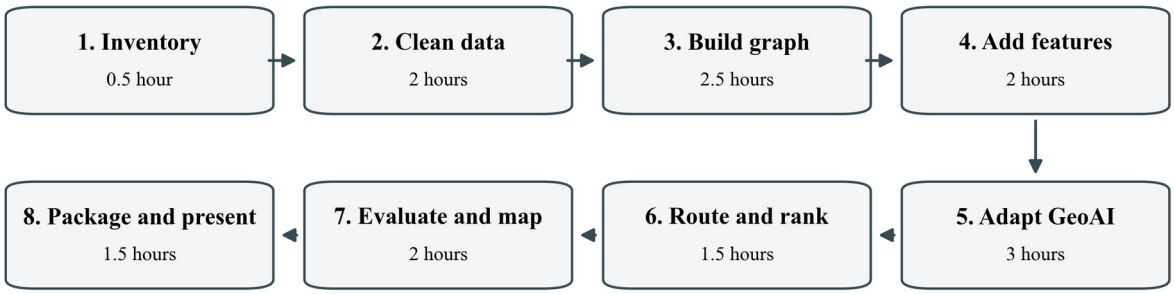
EmPath Roadmap and Delivery Checklist

Python prototype • Plano 75025, 75024, 75023 and 75074 • One week • 15–18 hours

Goal: compare residential access to temples, groceries and pharmacies using distance-only and environmental-friction routes.

Figure 1

Project pathway and time budget



15 hours of core work + up to 3 hours for troubleshooting and polish

Step / suggested day	Core time	Save before moving on
1. Inventory / Day 1	0.5 h	Sources + four-area coverage checklist
2. Clean data / Day 1	2 h	Clean spatial inputs + quality report
3. Build graph / Day 2	2.5 h	Graph + access connectors + audit
4. Add features / Day 3	2 h	Edge features + data dictionary
5. Adapt GeoAI / Day 4	3 h	Working model adapter + model card
6. Route and rank / Day 5	1.5 h	Routes + residential rankings
7. Evaluate and map / Day 6	2 h	Comparison table + interactive map
8. Package / Day 7	1.5 h	Run instructions + five-minute slides
Troubleshooting / as needed	Up to 3 h	Resolve blockers; polish after core works

Time boxes guide priorities; they do not guarantee completion. Reserve extra hours for missing data, graph repairs or model compatibility.

1 Inventory and Setup

☐ Open EmPath.code-workspace in VS Code; select .venv/bin/python and the EmPath Jupyter kernel.

☐ Read README.md, docs/CONFIRMED_SCOPE.md and docs/RELATED_PROJECTS.md. Reuse existing scripts and cleaned files.

☐ For every required layer, mark each ZIP: downloaded / partial / available online / unverified.

☐ Record source link, vintage, resolution, format, license/access terms, intended use and local file path.

Save: docs/DATA_SOURCES.md and docs/COVERAGE_CHECKLIST.md. Done: every required input has a status and next action.

2 Acquire and Clean Data

Input / priority	Source ID	Required check
Study boundary / required	D1	All four ZCTAs; distinguish ZIPs from ZCTAs
Walking network / required	D2 + OSM	Roads alone do not establish sidewalks or crossings
Residential origins / required	D3	Verify residential use and plausible entrances
Destinations / required	OSM	Temple, grocery and pharmacy; verify category/access
Edge canopy / required for canopy scenario	D4	Fine-scale coverage; NoData and 2021 vintage
Behavioral evidence / if used	R1	Understand route alternatives and choice labels
Neighborhood canopy / context	D5	Use for context, not as sidewalk shade

☐ Run .venv/bin/python scripts/clean_data.py --zipcodes 75025 75024 75023 75074; retain raw snapshots.

☐ Use GeoPandas, pandas, requests and Shapely: validate IDs/geometry, flag duplicates, preserve missing values; use EPSG:26914 for metric operations.

☐ Select roughly 5–10 verified residential origins per ZIP. Use OSMnx for destination candidates; verify names and access points. Missing OSM tags do not establish missing facilities.

Save: cleaned GeoPackages, origins.gpkg, destinations.gpkg and quality_report.md. Done: coverage and unresolved gaps are documented for all four areas.

3 Build and Audit the Walking Graph

☐ Use OSMnx graph_from_polygon(network_type="walk", retain_all=True); start with the four-ZCTA union plus roughly 500 m context. Expand if routes are truncated.

☐ Save the original GraphML. Inspect against Plano sidewalks and trails; do not turn sidewalk polygon boundaries into route edges.

☐ Attach origins/destinations at plausible pedestrian access points. Check nearest-node connectors for fences, inaccessible entrances and major-road crossings.

☐ Record connected components and disconnected origins. Preserve unreachable results; add trail connections only after checking topology, access and duplicates.

Tools: OSMnx, NetworkX, GeoPandas, Folium. References: Street Shadow (R2); UNA architecture (R5).

Save: walking_graph.graphml, access_connectors.gpkg, network_audit.md. Done: inspect at least one plausible route per ZIP.

4 Add Environmental Features

☐ Acquire a manageable clip of Texas 2021 canopy (D4); use Rasterio and NumPy to sample a documented edge corridor, initially a 10 m buffer.

☐ Store edge ID, length_m, canopy_fraction, valid_pixel_fraction, feature vintage and verified crossing constraints. Keep unknown values flagged.

☐ Check feature ranges and missingness by ZIP. Canopy is a proxy; do not describe it as time-specific shade or measured heat stress.

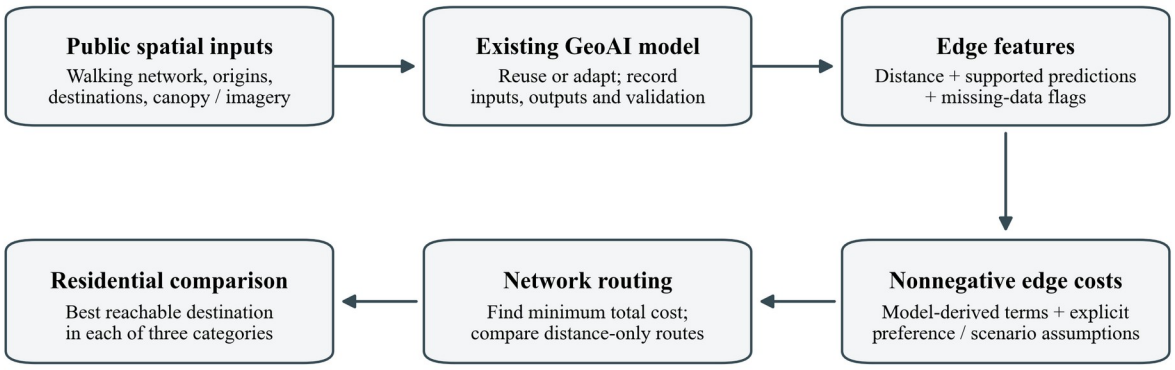
☐ Add traffic, weather, imperviousness or modeled shadows only after core features work. Neighborhood Tree Equity Score values remain contextual.

Save: edge_features.gpkg and docs/DATA_DICTIONARY.md. Done: each scoring feature has units, provenance and missingness checks.

5 Reuse and Adapt One GeoAI Component

Figure 2

Separate environmental prediction from routing and residential ranking



GeoAI predicts or estimates; routing selects paths. Keep the two components separate.

Option	Use when	Evidence to save
Existing environmental model	Runnable model + compatible public inputs are available	Model/version, prediction samples and local feature checks
Released route-choice analysis (R1)	Investigating shade–distance preferences	Reproduced notebook, parameters and transfer limitations
XGBoost or LightGBM adaptation	An existing suitable workflow and observed labels are available	Prediction target, inputs and held-out performance

☐ Choose ONE path. Reuse published code or pretrained weights; no new architecture or training from scratch is required.

☐ Write: model name/version; inputs; predicted quantity; original training geography; adaptation; role in edge costs; license.

☐ If using Singapore choices, reproduce the released analysis before modifying it; split by participant when fitting/evaluating. A choice model is behavioral calibration, not automatically a deep-learning model.

☐ Confirm the reused method meets the event’s GeoAI requirement. A precomputed ML canopy product is an input, not proof of a newly implemented AI model.

☐ Keep route-level predictions separate from additive edge costs. Do not call manually specified penalties learned weights; factors absent from observations cannot receive empirically learned preference weights.

Save: model_adapter.py or a model notebook and docs/MODEL_CARD.md. Done: the model runs, outputs are inspected, and reuse/adaptation is explicit. If blocked, record the GeoAI gap.

6 Calculate Routes and Rank Origins

☐ Define nonnegative additive edge costs. State units, model-derived terms and scenario assumptions. Applying a sunlight preference to canopy requires an explicit transfer assumption.

☐ Use NetworkX Dijkstra-based routing twice: distance-only baseline and environmental-friction costs.

☐ For each origin, select the lowest-cost reachable destination within each of the three categories. Destinations need not occur in every ZIP; retain surrounding network context.

☐ Rank by an equal-weight sum of the three category costs. Flag missing/unreachable categories; do not rank incomplete origins as fully accessible.

Tools: NetworkX, OSMnx, pandas, GeoPandas. References: Street Shadow (R2), CoolWalks (R3), Walk Score framing (R6).

Save: routes.gpkg and residential_rankings.csv with origin/ZIP, destination IDs, distances, category costs, total cost and reachability.

7 Evaluate and Build the Map

☐ Audit at least one route per ZIP and one major crossing; inspect connectors, access and suspicious jumps.

☐ Compare one baseline/alternative route pair: distance, canopy proxy and modeled cost. Test two plausible penalty settings; record ranking changes without forcing a preferred result.

☐ Evaluate model predictions separately. Performance on foreign observations does not establish accuracy for Plano pedestrians.

☐ Use Folium layer controls, markers, tooltips and popups for boundaries, origins, destination categories and route comparisons. Precompute results; use matplotlib for presentation figures.

☐ Use Streamlit only if interactive Python queries are essential and time remains. Prepare screenshots; basemap tiles can still require internet.

Save: output/empath_demo.html, evaluation.csv and docs/LIMITATIONS.md. Done: a viewer can see which origin ranks better and why. References: LA study (R4), City Health Dashboard (R6).

8 Package and Present

☐ Update README.md: installation, interpreter/kernel, execution order, data retrieval, source attribution and model/library versions.

☐ Verify the demo loads from a fresh session. Cache computed outputs and preserve a screenshot or recording backup.

☐ Prepare five-minute slides: problem/user → data/scope → graph/GeoAI → demo/finding → limitations/next steps.

☐ Disclose coverage gaps, proxy assumptions and geographic transfer. Environmental accessibility does not certify safety or medical suitability.

Final Handoff Checklist

Deliverable	Completion test
Sources + coverage checklist	Links, vintages, status and limitations for all four ZIP areas
Clean inputs + quality report	Raw provenance retained; geometry/schema gaps documented
Graph + edge features	Audited connections; stable IDs; missing values flagged
Model adapter + model card	Runs; reuse is attributed; target and limitations explained
Routes + rankings + evaluation	Three category costs; distance baseline; unreachable results retained
Demo + code + README	Loads reproducibly; dependencies and run sequence documented
Slides + backup	Fits five minutes; demonstration survives a live-download failure

Scope and Teamwork Rules

☐ Keep all four study areas. Reduce sampled origins, optional features or scenarios before expanding the model or interface.

☐ Use up to three additional hours for blockers and polish. Never replace missing evidence with fabricated values.

☐ Optional teammate: own either network/data QA or model evaluation; return a file and a documented check. ArcGIS Pro review is optional; exchange GeoPackage/GeoJSON.

☐ Future work: local validation, detailed crossings/traffic, time-dependent heat/shade and point-to-point lowest-friction navigation.

Source and Reference Key

Use the links below at the numbered steps. Existing project bibliography: docs/RELATED_PROJECTS.md and docs/resources.md.

ID	Source link	Use / limitation
D1	Census ZCTA boundaries	Study boundary; ZCTAs approximate postal ZIPs
D2	Plano sidewalk surfaces	Polygon inventory; not routing edges
D2	Plano trail centerlines	Candidate walking connections; verify access
D3	Plano address points	Origin candidates; verify residential use/entrance
D3	Plano parcels	Supporting geometry; not residential classification
D4	Texas 2021 canopy raster	10 m canopy proxy; substantial download may be needed
D5	Tree Equity Score	Neighborhood context, not sidewalk shade
OSM	OSMnx user reference	Walking graph and destination queries; local completeness varies

ID	Case study / implementation	What to reuse
R1	Melnikov et al. (2022), study	Observed shade–distance route choices; geographic transfer uncertain
R1	Singapore public data and code	Released analysis notebook; inspect data definitions
R2	Street Shadow Python repository	modules/graphs.py: shade fractions, weights and routing; manual preference parameter
R3	CoolWalks, Wolf et al. (2025)	Shade-sensitive costs and comparisons; code is Julia
R4	LA sidewalk heat study, Colaninno et al.	Homes-to-essentials heat/accessibility framing; full workflow uses QGIS
R5	UNA Python repository	Origin/destination architecture; optional reference, not another required dependency
R6	Walk Score methodology	Amenity-access score framing; not heat labels
R6	City Health Dashboard	Context and map communication; walkability uses Walk Score

Tool	Documentation	Purpose
NetworkX	Shortest paths	Weighted routing
Rasterio	Windowed raster reads	Feature extraction; requires readable raster, not just ZIP URL
Folium	Mapping user guide	Interactive HTML map

Core libraries: GeoPandas, pandas, Shapely, requests, NumPy, OSMnx, NetworkX, Rasterio, Folium and matplotlib. Add model-specific libraries only after selecting the model. Implementation remains Python-based.